

Dev AI Bootcamp — Workbook Bài tập Thực hành (track Dev) v1.0

Bổ trợ cho: Program (Phase 2 — Hands-on) · **track Dev (Developer)** theo §14.2.
Common Core: dùng chung tư duy/Delegation/Governance/Capstone với các track khác — file này chỉ **thay bộ bài tập + rubric + đầu ra Customer Zero** cho vai trò **lập trình viên**. **Đặc thù Dev:** đi **sâu nhất** về sinh/review code AI, test-gen, debug-with-AI, và **tự dựng Harness/agent**. **Công cụ xuyên suốt:** lái AI coding agent như orchestrator — nhập môn: Orchestrator Guide. **Thuật ngữ:** docs/GLOSSARY.md . **Cách dùng:** 6 bài chạy **trên CÙNG 1 case study** (hoặc 1 đề PB-01→06); cohort có thể **trộn nghề**. **Ngày:** 2026-07-02 · **Phiên bản:** v1.0

Changelog

Version	Ngày	Nội dung
v1.0	2026-07-02	Track Dev tách từ Workbook Common: giữ Common Core (case §0, telemetry, chống rubber-stamping); 6 bài Dev — Sinh code từ spec + review code AI · Test generation · Debug-with-AI · Tự dựng Harness/agent nhỏ · Feature/service chạy + test xanh (Customer Zero) · Delegation Map.

o. Case study chung — "Dự án LMS" (giống mọi track)

Dùng chung; hoặc thay bằng 1 đề **PB-01→PB-06** (PM-AI-Bootcamp-ProjectBriefs-v1.0.md).

Bối cảnh: Công ty 500 nhân viên quản lý **đăng ký nghỉ phép** bằng email + Excel → chậm, sai sót. Cần **Hệ thống Quản lý Nghỉ phép (LMS)** nội bộ.

Brief thô: "App xin nghỉ phép online, sắp duyệt trên điện thoại, thấy số ngày phép còn lại, HR báo cáo cuối tháng, ~2 tháng, tích hợp chấm công, bảo mật như app nội bộ khác." (chứa dữ liệu nhân sự)

Ràng buộc giả định: Team 3 dev (học viên) + 1 QA; mốc ~8 tuần; nội bộ, ~500 user.

💡 *AI sinh code rất nhanh — nhưng đọc-hiểu & chịu trách nhiệm code là việc của Dev. Cả track này chống thói "merge diff AI mà chưa đọc".*

Khung chung mọi bài tập (giống mọi track)

Mỗi bài gồm: **Mục tiêu năng lực** · **Thời lượng** · **Input** · **Các bước** · **Sản phẩm nộp** · **Telemetry bắt buộc** · **Rubric (0–4)** · **Prompt mẫu**.

Telemetry bắt buộc cho MỌI bài (thiếu = chưa hoàn thành): | Trường | Ghi gì | |---|---| | Tool dùng | tên tool/model | | Token (est) | tổng token in/out ước tính | | Thời gian | phút làm bài (gồm review) | | Số vòng lặp | bao nhiêu lần chỉnh prompt mới ra kết quả dùng được | | Rework | có phải làm lại code không (có/không + lý do) | | Dev-edit | ≥1 điểm Dev sửa lỗi/nấn code AI (mô tả ngắn + vì sao) |

Đo trung thực: thời gian là **giờ thật** (không suy từ token — sinh nhiều code ≠ chạy đúng); token **est** → **reconcile**; **Dev-edit** đo **phán đoán kỹ thuật của người**, không đo số dòng AI sinh.

DEV-EX-01 — Sinh code từ spec + review code AI (lỗi chống rubber-stamping)

- **Năng lực:** C1 AI Literacy · C2 Delegation · C4 Governance.
- **Thời lượng:** 120 phút.

Input: 1 user story + Acceptance Criteria (từ BA track, hoặc tự viết cho luồng "xin nghỉ phép").

Các bước: 1. AI sinh code cho 1 chức năng (vd API tạo đơn xin phép: validate → trừ quỹ phép → lưu) theo stack đã chọn. 2. **Dev review code AI như review PR người khác:** đọc từng phần, chú thích **bug / lỗi hổng / anti-pattern / thiếu edge case** (vd không khoá race khi trừ quỹ phép, không validate ngày quá khứ, lộ thông tin lỗi). 3. Dev **sửa & giải trình** ≥3 điểm; chạy lại cho qua AC. 4. **Hard-stop:** nếu spec không đủ để code đúng (vd "trừ quỹ phép" theo ngày làm việc hay ngày lịch?) → dừng, ghi câu hỏi, không để AI đoán.

Sản phẩm nộp: - Code chạy được + diff review (comment ≥3 vấn đề đã bắt). - REVIEW-NOTES.md : ≥3 điểm Dev sửa (bug/edge/anti-pattern) + lý do + ≥1 hard-stop. - Telemetry (kèm Dev-edit).

Rubric (0–4): | Điểm | Mô tả | |---|---| | 2 (Đạt) | Code chạy qua AC; Dev có review + sửa ≥3 điểm thật | | 3 | Bắt được bug/edge/lỗi hổng thực (race, validate, lộ lỗi); giải trình rõ; có hard-stop | | 4 | Review đạt chuẩn merge thật + rút ra checklist review code-AI tái dùng |

Prompt mẫu:

"Sinh API tạo đơn xin nghỉ phép theo stack [X]: validate input, trừ quỹ phép nhân viên, lưu DB, trả kết quả. Đúng theo Acceptance Criteria sau, nếu giả định thay vì bịa. AC: [...]" → Dev review như review PR: bắt race/edge/lỗi hổng, sửa, giải trình. KHÔNG merge khi chưa đọc hiểu.

DEV-EX-02 — Test generation + bắt "test giả"

- **Năng lực:** C1 · C4 · C6.
- **Thời lượng:** 90 phút.

Input: code từ DEV-EX-01 + AC.

Các bước: 1. AI sinh **unit + integration test** phủ AC, gồm **case biên & case lỗi** (hết phép, ngày quá khứ, duyệt trùng, huỷ sau duyệt). 2. **Dev soi test giả:** loại test `assert` rỗng / test luôn xanh / test không kiểm đúng hành vi / mock che mất bug thật. 3. Dev bổ test cho **nhánh AI bỏ sót**, chạy đo **coverage**, và **cố tình gài 1 bug** để chắc test **đỏ** đúng chỗ (kiểm test có thật sự bắt lỗi).

Sản phẩm nộp: - Bộ test chạy được + báo cáo coverage. - `TEST-NOTES.md` : ≥2 test giả đã loại + ≥2 nhánh bổ sung + kết quả "gài bug → test đỏ". - Telemetry.

Rubric (0–4): | Điểm | Mô tả | |---|---| | 2 (Đạt) | Test phủ AC chính, chạy được; có đo coverage | | 3 | Loại được test giả; phủ case biên/lỗi; chứng minh test bắt bug (gài bug → đỏ) | | 4 | Test đưa vào CI chạy tự động + gán ngưỡng coverage làm cổng |

Prompt mẫu:

"Sinh unit + integration test cho code sau, phủ Acceptance Criteria gồm case biên và case lỗi. Không viết test luôn xanh. Code & AC: [...]" → Dev soi test giả, gài bug thử, đo coverage.

DEV-EX-03 — Debug-with-AI (reproduce → root cause → fix)

- **Năng lực:** C1 · C3 · C6.
- **Thời lượng:** 75 phút.

Input (bug giả lập — phát kèm):

Báo lỗi: "Nhân viên xin 2 đơn phép cùng lúc, cả 2 được duyệt dù chỉ còn đủ quỹ cho 1

đơn → quỹ phép âm." (lỗi race/validate).

Các bước: 1. Dev + AI **tái hiện bug** (viết test đỏ tái hiện trước khi sửa). 2. AI đề xuất **giả thuyết root cause** — Dev **phản biện**: loại giả thuyết sai, xác nhận nguyên nhân thật bằng bằng chứng (log/test), **không sửa mù theo gợi ý đầu tiên**. 3. Fix + chạy test đỏ→xanh + kiểm không tạo hồi quy.

Sản phẩm nộp: - Test tái hiện bug (đỏ→xanh) + fix. - DEBUG-NOTES.md : giả thuyết đã loại + root cause thật + bằng chứng + vì sao không sửa mù. - Telemetry.

Rubric (0–4): | Điểm | Mô tả | |---|---| | 2 (Đạt) | Tái hiện được bug + fix làm test xanh | | 3 | Xác nhận root cause bằng bằng chứng, loại giả thuyết sai, không sửa mù | | 4 | Fix kèm test hồi quy + rút bài học phòng lớp lỗi tương tự |

Prompt mẫu:

"Có bug: 2 đơn phép cùng lúc đều được duyệt dù quỹ chỉ đủ 1 → quỹ âm. Đề xuất các giả thuyết root cause kèm cách kiểm chứng từng cái. Viết test tái hiện trước. Code: [...]"
→ Dev phản biện giả thuyết, xác nhận bằng bằng chứng rồi mới sửa.

DEV-EX-04 — Tự dựng Harness/agent nhỏ

- **Năng lực:** C2 Delegation · C3 · C4 — Harness là "bộ đồ nghề kỹ thuật" (Program §5), Dev đi sâu nhất.
- **Thời lượng:** 120 phút.

Input: 1 tác vụ lặp lại trong dự án (vd: sinh mã đơn, gắn nhãn đơn phép, tóm tắt log tuần).

Các bước: 1. Dựng **1 harness tối thiểu** quanh model cho tác vụ đó, đủ 3 mảnh cốt lõi: **context** (nạp đúng dữ liệu), **tool** (gọi hàm/API thật), **gate** (kiểm đầu ra trước khi dùng — fail-closed). 2. Cho agent chạy tác vụ **có ranh giới**: chỉ được làm trong phạm vi cho phép, **không** tự ghi DB thật khi chưa qua gate. 3. Dev đặt **cổng kiểm** (validate schema/định dạng, chặn khi không chắc) + log lại quyết định của agent (AgentOps mini).

Sản phẩm nộp: - Harness/agent chạy được (repo/link) + mô tả 3 mảnh context/tool/gate. - HARNESS-NOTES.md : ranh giới cho phép + cổng fail-closed + 1 ví dụ agent bị chặn đúng lúc. - Telemetry.

Rubric (0–4): | Điểm | Mô tả | |---|---| | 2 (Đạt) | Agent chạy được tác vụ với context + tool; có 1 cổng kiểm | | 3 | Gate fail-closed hoạt động (chặn output sai/không chắc); ranh giới rõ; có log quyết định | | 4 | Harness tái dùng cho tác vụ khác + có cơ chế người duyệt cho hành động rủi ro (BUILT-flagged) |

Prompt mẫu:

"Thiết kế một agent nhỏ tự [tác vụ], gồm: context nạp gì, tool nào được gọi, và cổng kiểm đầu ra (fail-closed) trước khi dùng. Agent KHÔNG được ghi dữ liệu thật khi chưa qua cổng. Mô tả ranh giới cho phép. Tác vụ: [...]" → Dev tự code harness + thử cho agent làm sai để xác nhận gate chặn.

DEV-EX-05 — Feature/service chạy + test xanh (Customer Zero mini của Dev)

- **Năng lực:** C1 · C3 · C6 — tự tay build để hiểu, chống rubber-stamping.
- **Thời lượng:** 150 phút.

Đầu ra Customer Zero của Dev (§14.2): feature/service chạy được + test xanh + telemetry, không dừng ở đoạn code lè.

Input: story + AC + (nếu có) ARCH / ADR từ track SA.

Các bước: 1. Build 1 lát cắt đọc chạy được end-to-end cho 1 luồng (UI/API → nghiệp vụ → DB), có test xanh (từ DEV-EX-02) và chạy được thật. 2. Gắn telemetry tối thiểu: log request, đo thời gian phản hồi, đếm lỗi — đủ để "nhìn" hệ thống chạy. 3. Ghi Dev Book mini ≥3 điểm AI sinh sai/thiếu và cách Dev sửa; đảm bảo CI/local chạy 1 lệnh lên được.

Sản phẩm nộp: - Feature/service chạy được (repo/link) + hướng dẫn chạy 1 lệnh + test xanh.
- SLICE-NOTES.md : ≥3 mục Dev Book (AI-sai / Dev-sửa) + ảnh/chỉ số telemetry cơ bản. - Telemetry bài tập (kèm Dev-edit).

Rubric (0–4): | Điểm | Mô tả | |---|---| | 2 (Đạt) | Lát cắt đọc chạy được + test xanh | | 3 | Chạy 1 lệnh lên được; có telemetry cơ bản; Dev Book ghi rõ ≥3 chỗ AI-sai/Dev-sửa | | 4 | Đủ chất lượng deploy nội bộ + telemetry đủ để quyết định vận hành |

Prompt mẫu:

"Build lát cắt đọc chạy được cho luồng 'xin nghỉ phép → duyệt' (UI/API → nghiệp vụ → DB) theo stack [X] và AC sau, kèm test và 1 lệnh chạy local. Thêm log/telemetry cơ bản. AC: [...]" → Dev tự chạy, sửa chỗ AI sai, ghi Dev Book.

DEV-EX-06 — Delegation Map + Autonomy Leash (Common Core, góc Dev)

- **Năng lực:** C2 AI Delegation · C4 Governance.

- **Thời lượng:** 75 phút.

Input: danh sách **việc code-level** (sinh boilerplate, viết test, refactor, fix bug, đổi schema DB, sửa phân quyền, viết migration...).

Các bước: 1. Lấy **≥15 việc code**. Mỗi việc điền: **nhóm 5-loại · mức L0–L5 · ai gỡ chốt duyệt** (AI tự / Dev review / bắt buộc người duyệt). 2. **Áp Leash A vs A+ ($A \approx L3, A+ \approx L4$):** sinh boilerplate/test nháp → A (AI tới bản nháp, **không tự merge**); **đổi schema DB / sửa phân quyền / migration dữ liệu nhân sự / đổi cấu hình bảo mật** → **A+ chờ duyệt (BUILT-flagged)** + qua **CI fail-closed** (test/lint/security phải xanh mới merge). 3. **Dev giải trình:** 1 dòng lý do mỗi việc $\geq L3$; chỉ ≥ 1 việc **cố giữ thấp** dù AI làm được (vd migration dữ liệu — sai là mất dữ liệu thật).

Sản phẩm nộp: - `DELEGATION-MAP-DEV.md` : bảng ≥ 15 việc $\times$ (nhóm · L0–L5 · ai duyệt · A/A+ · cổng CI · lý do). - Telemetry.

Rubric (0–4): | Điểm | Mô tả | |---|---| | 2 (Đạt) | ≥ 15 việc đủ nhóm + mức + ai duyệt; lý do hợp lý | | 3 | Việc chạm schema/phân quyền/migration đều A+ chờ duyệt + qua CI fail-closed; có việc cố giữ thấp | | 4 | Map gắn cổng CI cụ thể cho từng nhóm hazard + tái dùng làm quy ước merge của team |

Prompt mẫu:

"Với danh sách việc code sau, đề xuất mỗi việc: (a) giao AI/giữ người/AI-review/human-review/AI-không-nên; (b) mức L0–L5; (c) ai duyệt + cổng CI nào phải xanh. Đánh dấu việc chạm schema DB, phân quyền, migration dữ liệu nhân sự (không tự merge). Việc: [...]" → Dev phán xử lại, không để AI tự nâng mức.

Bảng ánh xạ bài tập ↔ năng lực (track Dev)

Bài	Trọng tâm Dev	Tuần gợi ý	Năng lực chính	Bắt buộc tốt nghiệp
DEV-EX-01	Sinh code + review code AI	W1	C1, C2, C4	✅ (lỗi)
DEV-EX-02	Test generation	W2	C1, C4, C6	✅
DEV-EX-03	Debug-with-AI	W2	C1, C3, C6	—
DEV-EX-04	Tự dựng Harness/agent	W3	C2, C3, C4	— (khuyến nghị mạnh)
DEV-EX-05	Feature/service chạy + test xanh (Customer Zero)	W4	C1, C3, C6	✅
DEV-EX-06	Delegation Map	W3	C2, C4	— (khuyến nghị mạnh)

Trọng số dịch về (theo §14.2): code chạy & chất lượng **nặng**; tài liệu nhẹ. Đầu ra Customer Zero = feature/service chạy + test xanh + telemetry.

Hướng dẫn giảng viên (facilitator notes) (giống mọi track)

- **Chấm cái gì:** chấm **phán đoán kỹ thuật của Dev** (bug bắt được khi review, test giả loại được, root cause xác nhận bằng bằng chứng) — không chấm số dòng AI sinh. Cột `Dev-edit` là bằng chứng quan trọng nhất.
- **Tiêu chí trượt — "rubber-stamping":** **merge code AI mà không đọc hiểu / không có dấu vết review** → tối đa 1 điểm dù "chạy được". Đây là thói nguy nhất của track Dev — cả khoá phải diệt.
- **Bẫy cổ ý:** race khi trừ quỹ phép & thiếu validate (EX-01), test luôn xanh / assert rỗng (EX-02), sửa mù theo gợi ý đầu (EX-03), agent tự ghi DB không qua gate (EX-04), lát cắt "chạy trên máy em" nhưng CI đỏ (EX-05).
- **Cohort trộn nghề:** Dev nhận `STORIES` / AC từ BA, `ARCH` / `ADR` từ SA → build slice thật → mô phỏng bàn giao & phối hợp team.

Track Dev thuộc bộ Workbook nhiều-nghề dùng chung Common Core. Xem thêm track BA (`...Workbook-BA-...`), SA (`...Workbook-SA-...`), PM (`...Workbook-Common-...`).